Core Knowledge Series: **Hands-On with R**

Session One: **Introduction and Data Manipulation**

Bioinformatics Core (BSR)

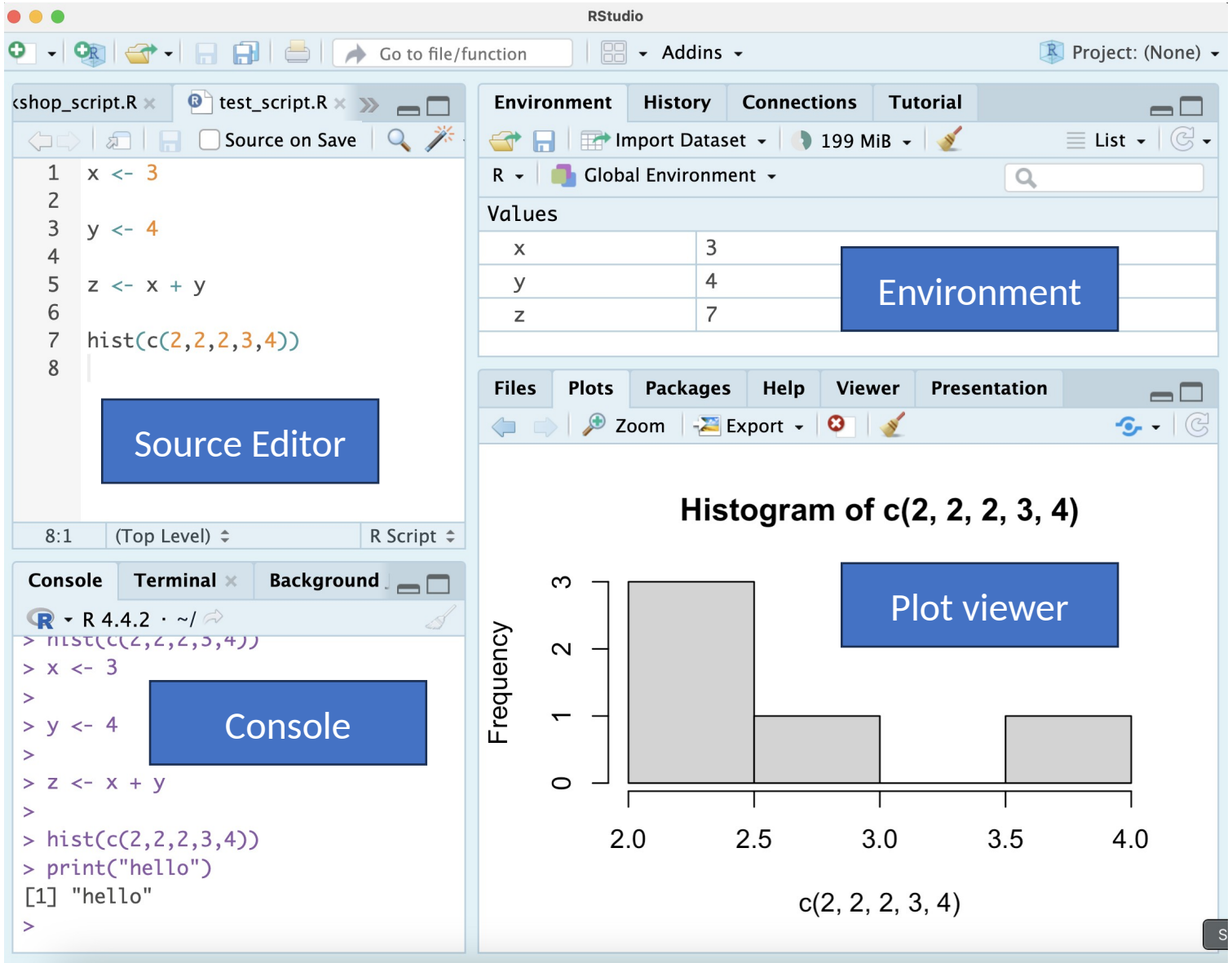
Cold Spring Harbor Laboratory

R Studio and R

- R is a powerful open-source programming language specifically designed for statistical computing, data analysis, and visualization. Many bioinformatics tools are developed in R.
- RStudio is user-friendly and makes coding in R easier with features like debugging, plotting, and environment.
- Packages native to R
  - Bioinformatics tools:
    - DESeq2 for RNA-Seq
    - Seurat for single-cell
    - Diffbind for ChIP and ATAC-seq
  - Data Manipulation
    - dplyr, tibble, tidyr, tidyverse

Cold Spring Harbor Laboratory

R Studio



- Source Editor (Top-Left)
  - Writing/editing R and Rmd scripts
- Environment (Top-Right)
  - Stores all created variables
- Console/Terminal (Bottom-Left)
  - Console: Executes R commands
  - Terminal: Navigate through files
- Files/Plots/Help (Bottom-right)
  - Mainly used for plot viewing. Can also navigate files and display help for functions.

Cold Spring Harbor Laboratory

Libraries in R

```
install.packages("Seurat")
library(Seurat)
```

- **Libraries** in R are collections of functions, datasets, and documentation that extend R's capabilities.
  - dplyr – data manipulation
  - ggplot2 – customized plotting
- Libraries must be **installed** and **loaded** before use; installation is done with: **install.packages("package_name”)** loading is done with: **library(package_name)**

Basic R Commands

```r
# Printing
print("Hello World")

# Variable assignment and operations
x <- 5  # Integer
y <- 3  # Integer

# Arithmetic operations
print(x + y)  # Addition
print(x * y)  # Multiplication
print(x ^ y)  # Exponentiation
print(x / y)  # Division

# Assigning result to a new variable
z <- x + y
print(z)

# Logical comparisons
print(x > y)  # Greater than
print(x == y)  # Equal to
print(x != y)  # Not equal to

# Combining variables into a vector
all_variables <- c(x, y, z)
print(all_variables)
```

- **print** function
- To run a line of code, move cursor to specific line, hold command, and press enter
- Variable assignment with “<-”
- Performing arithmetic operations
- Assigning variables to variables
- Logical comparisons – what do they return/print?
- How to make a vector of variables. What is a vector of variables?

Cold Spring Harbor Laboratory

Practice

- Create variables a and b
  - a is 7
  - b is 9
- c is the product of a and b
- d is a to the power of b
- e is c minus d
- Store a,b,c,d,e in a vector called all_numbers

Solution

```r
a <- 7

b <- 9

c <- a * b

d <- a ^ b

e <- c - d

all_numbers <- c(a,b,c,d,e)
```

Cold Spring Harbor Laboratory

Types of variables in R

```r
# Different types of variables
int_var <- 10L  # Integer
num_var <- 10.5  # Numeric (double)
char_var <- "Hello"  # Character
bool_var <- TRUE  # Logical (Boolean)

# Checking variable class
print(class(int_var))
print(class(num_var))
print(class(char_var))
print(class(bool_var))

# Vectors (One-dimensional data structures)
vec_num <- c(1, 2, 3, 4, 5)  # Numeric vector
vec_char <- c("A", "B", "C")  # Character vector
vec_logical <- c(TRUE, FALSE, TRUE)  # Logical vector

# Lists (Heterogeneous data structures)
list_var <- list(int_var, num_var, char_var, bool_var)
print(list_var)

# Matrices (Two-dimensional numeric structures)
mat <- matrix(1:9, nrow = 3, ncol = 3)
print(mat)


# Data frames (Tabular structures)
count_data <- data.frame(
  Gene = c("Gene1", "Gene2"),
  Sample1 = c(100, 200),
  Sample2 = c(150, 250)
)
print(count_data)
```

- Numeric variables
  - Integers – Whole numbers. L specifies integer.
  - Numeric/double – Real numbers with decimal point
- Character variables
  - Text or string data
- Logical variables
  - TRUE or FALSE
- Data structures
  - Vectors – collection of same type of elements
  - Lists – collection of same or different types of elements
  - Matrix – 2D collection of same type of elements
  - Data Frame – 2D collection of same or different types of data – built for data manipulation tools

Practice

- Create a data frame called ages_df with 2 columns and 2 rows
  - Name: Bill, Alice
  - Age: 5, 42
- Store the ages in a variable called ages
  - Hint: The columns can be accessed using the “$” operator (object_name$column_name)
- Print the class(type) of ages

Solution

```
1  ages_df <- data.frame(
2    Name = c("Bill", "Alice"),
3    Age = c(5, 42))
4
5  ages <- ages_df$Age
6
7  print(class(ages))
```

Functions in R

```
> ?print
```

Description

`print` prints its argument and returns it *invisibly* (via `invisible(x)`). It is a generic function which means that new printing methods can be easily added for new `class`es.

Usage

```
print(x, ...)

## S3 method for class 'factor'
print(x, quote = FALSE, max.levels = NULL,
      width = getOption("width"), ...)

## S3 method for class 'table'
print(x, digits = getOption("digits"), quote = FALSE,
      na.print = "", zero.print = "0",
      right = is.numeric(x) || is.complex(x),
      justify = "none", ...)

## S3 method for class 'function'
print(x, useSource = TRUE, ...)
```

Arguments

`x`	an object used to select a method.
`...`	further arguments passed to or from other methods.
`quote`	logical, indicating whether or not strings should be printed with surrounding quotes.
`max.levels`	integer, indicating how many levels should be printed for a factor; if `0`, no extra "Levels" line will be printed. The default, `NULL`, entails choosing `max.levels` such that the levels print on one line of width `width`.
`width`	only used when `max.levels` is NULL, see above.
`digits`	minimal number of *significant* digits, see `print.default`.
`na.print`	character string (or `NULL`) indicating `NA` values in printed output, see `print.default`.
`zero.print`	character specifying how zeros (0) should be printed; for sparse tables, using `"."` can produce more readable results, similar to printing sparse matrices in **Matrix**.
`right`	logical, indicating whether or not strings should be right aligned.
`justify`	character indicating if strings should left- or right-justified or left alone, passed to `format`.
`useSource`	logical indicating if internally stored source should be used for printing when present, e.g., if `options(keep.source = TRUE)` has been in use.

- Functions are reusable code: take inputs (arguments) and return an output
- Arguments control behavior: use defaults or name them in any order
- "?" Before a function will give documentation for that function
  - ?print

Cold Spring Harbor Laboratory

Practice

- Print the “CSHL” with surrounding quotes

Solution

```
> print("CSHL", quote = TRUE)
[1] "CSHL"
```

Reading in example counts data

```
counts <- read.csv(url)

str(counts)

# Preview the data
print("Data Preview:")
print(head(counts))
```

Source Editor

```
> str(counts)
'data.frame':   50 obs. of  7 variables:
 $ Gene    : chr  "TP53" "EGFR" "MYC" "KRAS" ...
 $ Tumor_1 : int  92 107 101 130 106 113 109 118 87 91 ...
 $ Tumor_2 : int  114 94 101 100 98 95 136 123 105 127 ...
 $ Tumor_3 : int  111 111 114 124 92 89 120 102 93 106 ...
 $ Normal_1: int  89 85 89 69 99 100 71 86 103 93 ...
 $ Normal_2: int  100 80 82 68 99 88 91 75 107 88 ...
 $ Normal_3: int  104 100 97 90 111 81 109 96 80 93 ...
```

Console

```
[1] "Data Preview:"
> print(head(counts))
    Gene Tumor_1 Tumor_2 Tumor_3 Normal_1 Normal_2 Normal_3
1   TP53      92     114     111       89      100      104
2   EGFR     107      94     111       85       80      100
3    MYC     101     101     114       89       82       97
4   KRAS     130     100     124       69       68       90
5 PIK3CA     106      98      92       99       99      111
6    MET     113      95      89      100       88       81
```

- **read.csv** function will read in a csv file in R
- **str** gives the structure of more complex objects like data frames
- **head** will show the first few rows of the data frame
- This data frame has 7 columns. 1 column identifying the gene, 6 with sample identifiers. 50 rows, each representing one gene.

Should the gene name be a column?

```r
# Convert gene column to row names
rownames(counts) <- counts$Gene
counts <- counts[, -1]  # Remove the Gene column

counts <- read.csv(url, row.names = 1)
```

```
> str(counts)
'data.frame':   50 obs. of  6 variables:
 $ Tumor_1 : int  92 107 101 130 106 113 109 118 87 91 ...
 $ Tumor_2 : int  114 94 101 100 98 95 136 123 105 127 ...
 $ Tumor_3 : int  111 111 114 124 92 89 120 102 93 106 ...
 $ Normal_1: int  89 85 89 69 99 100 71 86 103 93 ...
 $ Normal_2: int  100 80 82 68 99 88 91 75 107 88 ...
 $ Normal_3: int  104 100 97 90 111 81 109 96 80 93 ...
```

- Data frames can have **row names,** similar to indices in python.
- The **rownames** function will set the row names of a data frame.
- The **$** operator accesses a column in a data frame (eg. counts$Gene)
- counts[,-1] specifies to remove the first column (Gene column)

Performing CPM normalization on raw counts

```r
# Step 3: CPM Normalization
# Calculate the total counts per sample
total_counts <- colSums(counts)
total_counts
# Divide each count by the total counts for its column and multiply by 1e6
cpm <- counts / total_counts * 1e6

# Write the new file to a csv
write.csv(cpm, "/Users/rouse/CSH/R_basics/data/Tumor_vs_Normal_counts_CPM.csv")
```

- Let's do some calculations
- **Counts per million** helps to normalize for sample sequencing depth.
- Calculation:
  - calculate the total counts
  - divide raw counts by total counts
  - multiply by 1 million
- colSums will calculate the sum of all columns in a data frame
- Often want to write normalized counts to a new file

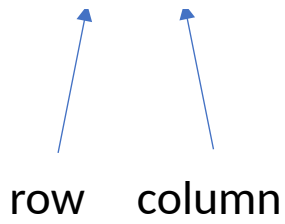
Cold Spring Harbor Laboratory

Data frame indexing

```
# Indexing data frames
cpm[, 1]
cpm [1,]

cpm[, c(1:3)]
cpm[, c(4:6)]

cpm[1,1]
```



- Columns and rows can be reached using **brackets** after the data frame []
- **Comma** separates row index from column index with row index coming first
- c(1:3) will give a vector from 1-3. Same as c(1,2,3)

Cold Spring Harbor Laboratory

Practice

- f is the value in the first row and second column of cpm dataframe
- g is the value in the second row and third column of cpm dataframe
- h is f divided by g
- i is the first column of cpm
- j is the first column of cpm multiplied by h

Solution

```r
f <- cpm[1, 2]

# g: value in second row, third column
g <- cpm[2, 3]

# h: f / g
h <- f / g

# i: first column
i <- cpm[, 1]

# j: first column multiplied by h
j <- i * h

```

14:1 (Top Level) R Script

Console Terminal Background Jobs

R 4.5.0 · ~/

```
> # Output
> f
[1] 23331.97
> g
[1] 22107.15
> h
[1] 1.055404
> i
 [1] 18764.02 21430.00 20671.31 25778.31 21810.70 22505.48 22231.29 23633.09 17805.98 18044.81
24897.12
[12] 19518.02 18356.11 15421.59 15963.98 19432.88 17489.71 20713.01 17540.28 18025.24 23741.30
18441.40
[23] 17489.71 18721.37 17948.19 19827.76 15963.98 18044.81 18518.52 19318.86 22027.33 23432.81
22513.30
[34] 18441.40 22427.98 19518.02 21007.55 15621.87 20057.31 18044.81 21193.42 16530.57 19987.76
16422.99
[45] 19647.97 19036.29 21193.42 24696.28 21211.50 19427.20
```

Calculating averages using data frame indexing

```
Tumor_Mean <- rowMeans(cpm[, c(1:3)])
Normal_Mean <- rowMeans(cpm[, c(4:6)])
```

- rowMeans will calculate the means for the specified columns in each row.

Practice

- Extract the name of the first gene in the cpm data frame
  - rownames() function will give you a list of all rownames
  - [] can be used to index a list/vector
    - Vector[1] will give the first element
- Determine whether the tumor mean or normal mean is higher for this gene
  - Use Tumor_Mean and Normal_Mean
  - [] to index each

Solution

```
names <- rownames(cpm)
names[1]

Tumor_Mean[1] > Normal_Mean[1]
```

Creating a data frame

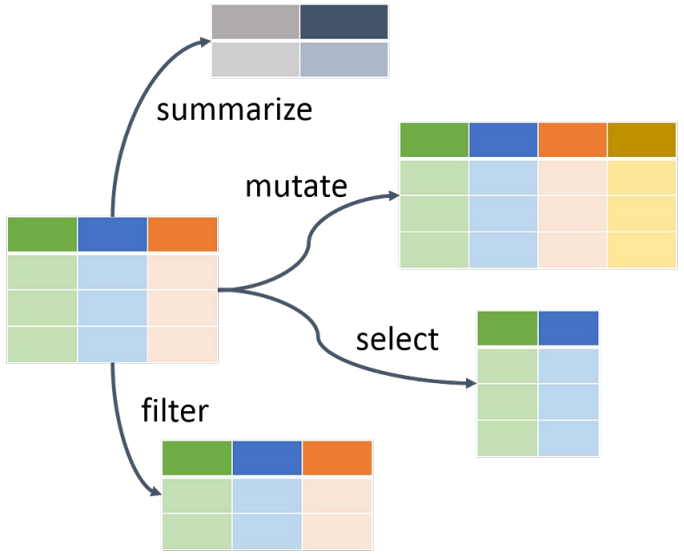
```
## Creating data frame with calculated averages
combined_avg <- data.frame(
  Gene = rownames(cpm),
  Tumor_Mean = Tumor_Mean,
  Normal_Mean = Normal_Mean
)
```

- data.frame function is used to create data frames.
  - column_name = c(values…)

Cold Spring Harbor Laboratory

dplyr

- dplyr is the main tool used for data manipulation in R
  - Similar to SQL
- dplyr uses the %>% (“pipe”) operator to pass data frames to a function
- The main goal of dplyr is to perform functions and and rearrange data frames



Cold Spring Harbor Laboratory

Simple dplyr functions - select

```
library(dplyr)
tumor_data <- combined_avg %>%
  select(Gene, Tumor_Mean)
```

Note: THESE ARE THE SAME

```
tumor_data <- select(combined_avg, Gene, Tumor_Mean)
```

- dplyr is the main tool used for data manipulation in R
  - Similar to SQL
- dplyr uses the %>% ("pipe") operator to pass data frames to a function
- Pipe is used since it is often cleaner when doing multiple functions at one time
- **select** function is used to select certain columns

Cold Spring Harbor Laboratory

Simple dplyr functions - mutate

```
### MUTATE
mutated_data <- combined_avg %>%
  mutate(Difference = Tumor_Mean - Normal_Mean)

head(mutated_data)
```

```
> head(mutated_data)
         Gene Tumor_Mean Normal_Mean Difference
TP53     TP53   21645.17    20005.99  1639.1763
EGFR     EGFR   20725.62    17601.25  3124.3642
MYC       MYC   21568.09    18290.51  3277.5756
KRAS     KRAS   23509.81    15083.56  8426.2468
PIK3CA PIK3CA   20209.26    21093.35  -884.0963
MET       MET   19726.78    17867.63  1859.1486
```

- **mutate** function is used to create a new column that are often functions of existing variables

Cold Spring Harbor Laboratory

Simple dplyr functions - filter

```
##### FILTER
filtered_data <- combined_avg %>%
  filter(abs(Tumor_Mean - Normal_Mean) > 2000)

nrow(filtered_data)
```

- **filter** function is used to subset a data frame based on certain conditions
- **abs** calculates absolute value
- **nrow** gives the number of rows in the data frame. Number of genes that remain after filtering

Practice

- Filter rows in cpm data frame
  - Keep only the genes where the average CPM across all samples is greater than 10,00
  - *Hint:* Use filter() with rowMeans() on cpm to keep only rows above the threshold
- Add a new column Log2_Tumor_Mean
  - This column should be the log2-transformed mean CPM of the Tumor samples (columns 1–3)
  - *Hint:* Use mutate() and calculate rowMeans(cpm[, 1:3]), then wrap it in log2()

Solution

```
cpm %>%
  filter(rowMeans(cpm) > 10000) %>%
  mutate(Log2_Tumor_Mean = log2(rowMeans(cpm[, 1:3])))
```

Joining data

```
####### Joining
# Example meta data 1
meta_data_1 <- data.frame(
  Sample = c("Normal_1", "Normal_2", "Normal_3", "Tumor_1"),
  Group = c("Normal", "Normal", "Normal", "Tumor"),
  Condition = c("Healthy", "Healthy", "Healthy", "Cancer")
)

# Example meta data 2
meta_data_2 <- data.frame(
  Sample = c("Normal_2", "Tumor_1", "Tumor_2", "Tumor_3"),
  Batch = c("A", "B", "B", "C")
)
```

	Sample	Group	Condition
1	Normal_1	Normal	Healthy
2	Normal_2	Normal	Healthy
3	Normal_3	Normal	Healthy
4	Tumor_1	Tumor	Cancer

	Sample	Batch
1	Normal_2	A
2	Tumor_1	B
3	Tumor_2	B
4	Tumor_3	C

Left joining

```
# Left join: Keep only all rows in first data frame mentioned and add columns that
# match from the second
left_join_result <- left_join(meta_data_1, meta_data_2, by = "Sample")
```

meta_data_1

	Sample	Group	Condition
1	Normal_1	Normal	Healthy
2	Normal_2	Normal	Healthy
3	Normal_3	Normal	Healthy
4	Tumor_1	Tumor	Cancer

meta_data_2

	Sample	Batch
1	Normal_2	A
2	Tumor_1	B
3	Tumor_2	B
4	Tumor_3	C

left_join_result

	Sample	Group	Condition	Batch
1	Normal_1	Normal	Healthy	*NA*
2	Normal_2	Normal	Healthy	A
3	Normal_3	Normal	Healthy	*NA*
4	Tumor_1	Tumor	Cancer	B

- Left joining merges two data frames
  - keeps all rows from the first data frame
  - matching the two data frames by specific column(s)
  - adding columns from second data frame to the first data frame
  - Will return NA for samples where there is no match

Full/Outer joining

```
# Takes all samples and joins all information
full_join_result <- full_join(meta_data_1, meta_data_2, by = "Sample")
```

meta_data_1

	Sample	Group	Condition
1	Normal_1	Normal	Healthy
2	Normal_2	Normal	Healthy
3	Normal_3	Normal	Healthy
4	Tumor_1	Tumor	Cancer

meta_data_2

	Sample	Batch
1	Normal_2	A
2	Tumor_1	B
3	Tumor_2	B
4	Tumor_3	C

full_join_result

	Sample	Group	Condition	Batch
1	Normal_1	Normal	Healthy	*NA*
2	Normal_2	Normal	Healthy	A
3	Normal_3	Normal	Healthy	*NA*
4	Tumor_1	Tumor	Cancer	B
5	Tumor_2	*NA*	*NA*	B
6	Tumor_3	*NA*	*NA*	C

- Full joining merges two data frames
  - keeps all samples from both data frames
  - matching the two data frames by specific column(s)
  - adding columns from second data frame to the first data frame
  - Will return NA for samples where there is no match

Cold Spring Harbor Laboratory

Inner joining

```
# Outputs only samples that are present in both data frames
inner_join_result <- inner_join(meta_data_1, meta_data_2, by = "Sample")
```

meta_data_1

	Sample	Group	Condition
1	Normal_1	Normal	Healthy
2	Normal_2	Normal	Healthy
3	Normal_3	Normal	Healthy
4	Tumor_1	Tumor	Cancer

meta_data_2

	Sample	Batch
1	Normal_2	A
2	Tumor_1	B
3	Tumor_2	B
4	Tumor_3	C

inner_join_result

	Sample	Group	Condition	Batch
1	Normal_2	Normal	Healthy	A
2	Tumor_1	Tumor	Cancer	B

- Inner joining merges two data frames
  - keeps only samples that are present in both data frames
  - matching the two data frames by specific column(s)
  - adding columns from second data frame to the first data frame

Cold Spring Harbor Laboratory

Test your knowledge

Step 1 — Create a new data table
In R, create a data frame called city_data with the following columns and values:

- **City**: "Cold Spring Harbor", "Huntington", "Syosset", "Melville", "Bayshore”
- **Population**: 153000, 845000, 473000, 920000, 267000
- **Avg_Temp**: 12.4, 19.6, 22.1, 25.3, 15.1
- **Coastal**: TRUE, FALSE, FALSE, FALSE, TRUE

Cold Spring Harbor Laboratory

Solution

```r
city_data <- data.frame(
  City       = c("Cold Spring Harbor", "Huntington", "Syosset", "Melville", "Bayshore"),
  Population = c(153000, 845000, 473000, 920000, 267000),
  Avg_Temp   = c(12.4, 19.6, 22.1, 25.3, 15.1),
  Coastal    = c(TRUE, FALSE, FALSE, FALSE, TRUE)
)
```

Questions

- **Filtering**
  Using dplyr's filter, keep only the cities where:
  - Coastal == TRUE **AND** Avg_Temp > 15
- **Mutating**
  Add two new columns to city_data:
  - Temp_F = (Avg_Temp * 9/5) + 32
  - Pop_in_Millions = Population / 1e6
- **Summary Stats**
  - Find the mean population across all 5 cities.
  - Find the city with the highest average temperature.
    - Hint: Dplyr has an arrange function, arrange(desc(Avg_Temp))
    - Hint: Dplyr also has a slice funciont slice(1) will give you the first value
- **Join**
  Create a second data frame called city_region with the following:
  - **City**: "Cold Spring Harbor", "Syosset", "Melville", "Newhaven"
  - **Region**: "North", "East", "South", "Central"
  - Perform a full join with city_data by "City".

CSH Cold Spring Harbor Laboratory

Solution

```r
# ---- 1) Filtering: Coastal == TRUE AND Avg_Temp > 15 ----
filtered_cities <- city_data %>%
  filter(Coastal == TRUE, Avg_Temp > 15)
filtered_cities


# ---- 2) Mutating: Temp_F and Pop_in_Millions ----
city_data_aug <- city_data %>%
  mutate(
    Temp_F = (Avg_Temp * 9/5) + 32,
    Pop_in_Millions = Population / 1e6
  )
head(city_data_aug)

# ---- 3) Summary Stats ----
mean_population <- mean(city_data$Population)
mean_population

hottest_city <- city_data %>%
  arrange(desc(Avg_Temp)) %>%
  slice(1) %>%
  select(City)

hottest_city

# ---- 4) Full Join with city_region ----
city_region <- data.frame(
  City   = c("Cold Spring Harbor", "Syosset", "Melville", "Newhaven"),
  Region = c("North", "East", "South", "Central")
)

full_join_result <- full_join(city_data_aug, city_region, by = "City")
full_join_result
```

Cold Spring Harbor Laboratory

Additional Practice Questions

- **Filtering with OR**
  Using dplyr::filter, return all cities where:
  - Coastal == TRUE **OR** Population > 800000
- **Selecting Columns**
  Using dplyr::select, create a new data frame with only City and Avg_Temp from city_data.
- **Sorting by Population**
  Using dplyr::arrange, sort city_data in **descending** order of Population. Show only the first 3 rows.
- **Average Temperature for Coastal Cities**
  Using dplyr, calculate the **mean Avg_Temp** for only cities where Coastal == TRUE.
  - Hint: Use summarise dplyr function.
  - Inside summarise(), you name the new column you want to create (e.g., Mean_Avg_Temp = …) and then provide a calculation.
  - You can use any summary function like mean(), sum(), min(), or max().
  - When used with filter() first, summarise() will only calculate the summary on the filtered rows.

```r
# ---- Filtering for coastal and population
q5_result <- city_data %>%
  filter(Coastal == TRUE | Population > 800000)
q5_result

# ----7_ Selecting city and temperature columns ----
q6_temp <- city_data %>%
  select(City, Avg_Temp)
q6_temp

# ----7) Sorting by Population (top 3) ----
q7_top3 <- city_data_aug %>%
  arrange(desc(Population)) %>%
  head(3)
q7_top3

# ---- 8) Average Temperature for Coastal Cities ----
q8_mean_coastal_temp <- city_data %>%
  filter(Coastal == TRUE) %>%
  summarise(Mean_Avg_Temp = mean(Avg_Temp))
q8_mean_coastal_temp

q8_coastal <- city_data %>%
  filter(Coastal == TRUE)

mean(q8_coastal$Avg_Temp)
```

Resources

BiTS
Bioinformatics Social

Join us !

- Connect with CSHL bioinformaticians
- Share analysis experiences
- Regular best-practice guidelines
- Explore novel tools & pipelines
- GitHub & Teams community

CSH Cold Spring Harbor Laboratory